Compte rendu Projet de Fin d'Etudes (PFE) : Système Laser Game Connecté

Sommaire :

1. Introduction au Projet 5
1.1 Gestion du projet 6
1.2 Outils utilisés 7
2. Description du système 8
2.1 Système de « tir » (émission – réception)..... 9
2.1.1Technologies utilisées 9
Émission - Réception LASER..... 10
Émission - Réception IR..... 10
Conclusion 11
2.2 Implémentation de l'émission et la réception 12
2.2.1 Hardware .....**Erreur ! Signet non défini.**
2.2.2 Protocole NEC .....**Erreur ! Signet non défini.**
2.3 Gestion des lumières.....**Erreur ! Signet non défini.**
2.4 Serveur pour le suivi de score.....**Erreur ! Signet non défini.**
2.4.1 Côté Carte.....**Erreur ! Signet non défini.**
2.4.2 Côté serveur .....**Erreur ! Signet non défini.**
4. Améliorations possible.....**Erreur ! Signet non défini.**
4.1 Conception de carte .....**Erreur ! Signet non défini.**
4.2 Modélisation 3D .....**Erreur ! Signet non défini.**
5. Sources .....**Erreur ! Signet non défini.**

Table des figures :

Figure 1 - nRF 9151 - DK 5
Figure 2 - Composants Lasers..... 6
Figure - Composants IR..... 7
Figure - Circuits IR 8
Figure - NEC : exemple modulation par largeur d'impulsion 9
Figure - Exemple de trame..... 9
Figure - LEDs RGB SK6812 10
Figure - Protocole d'envoie des donnés pour la SK6812..... 10
Figure 9 - Schéma de LED à cathode commune 11
Figure 10 - Circuit contrôle de LED RGB 12

1. Introduction au Projet

Ce projet de fin d'étude a été proposé par CODIUM Electronique. Il consiste à développer un système de « laser game » complet, intégrant des composants électroniques embarqués pour les plastrons, les modules d'épaules et le pistolet, avec une connectivité sans fil NR+.

L'objectif principal est de concevoir de A à Z une solution technique fonctionnelle capable de détecter les tirs, d'identifier les tireurs et de transmettre les données de jeu en temps réel à un serveur.

Ce projet peut se décomposer en trois parties majeures :

- **La recherche et le développement** d'une technologie de « tir » (émission/réception) adaptée.
- **La programmation de LEDs** particulières pour réagir aux tirs reçus.
- **Le développement d'un serveur HTTP** et de l'envoi d'informations depuis les cartes vers ce serveur, via la technologie NR+ (finalement LTE-M).

Ce projet a fait appel à des compétences en électronique, mais surtout en programmation pour l'embarqué, à des études de documentation et à une réflexion sur la conception électronique.

1.1 Gestion du projet

On organise le projet autour du planning donné dans le sujet de PFE. (Pour rappel) il s'organise autour de quatre mois : d'octobre 2025 à janvier 2026, et suit les grandes étapes décrites ci-dessous :

- **Octobre 2025 :** L'analyse théorique et les tests du système de tir : on commence par se renseigner sur ce qui existe déjà en termes de système de laser game et les technologies qu'ils utilisent. Puis on commence à faire des tests avec le matériel à notre disposition et la documentation que l'on peut trouver en ligne, pour savoir quelle technologie choisir pour notre système.

- **Novembre 2025 :** Choix de la technologie, prototypage et tests de performances : une fois l'analyse théorique complète, on fait le choix de la technologie ainsi que du protocole à utiliser. On commande ainsi les composants nécessaires et on les implémente afin d'y conduire des tests. Pour le système de tir, des tests de portées et de directivité sont nécessaires.

- **Décembre 2025 :** Début de la programmation embarquée : après avoir une émission fonctionnelle, on commence la phase de programmation embarquée

sur microcontrôleur. L'objectif ici est de faire réagir le système à la réception d'un tir afin de, dans un premier temps, gérer les LEDs de l'équipement puis, dans un second temps, d'envoyer les informations récupérées à la réception vers un serveur HTTP via le réseau mobile.

- **Janvier 2026 :** Finalisation de la programmation embarquée : mise en commun de toutes les parties du projet, tests de fonctionnement complet. On assemble les différentes parties du projet et on les teste ensemble afin d'obtenir un prototype fonctionnel.

La phase de conception, de routage et de fabrication d'un prototype hardware a été retirée du planning à la suite de la décision de mon tuteur de PFE. N'ayant qu'un total de 14 semaines (16 semaines en comptant le congé de Noël) et pouvant compter 8 semaines entre la commande et la réception de ces cartes prototypes, il a été préférable de se focaliser sur l'aspect implémentation software des différentes fonctionnalités. Cette phase ainsi que la modélisation 3D de l'équipement étant donc laissées de côté si ce projet est repris ultérieurement.

Pour rappel, nous avions « seulement » 28 jours étalés sur 14 semaines, dédiés à ce projet. Il était difficile de fournir une solution complète et fonctionnelle en plus d'un prototype de carte correctement routée et prête à l'emploi. Un travail à temps plein aurait été adapté afin de fournir une solution complète un peu moins « expérimental » que celle proposée dans ce rapport.

1.2 Outils utilisés

Afin de développer une solution et de programmer un prototype software sur le kit de développement du nRF9151 (nRF9151-DK) de Nordic Semiconductor, nous utilisons Visual Studio Code avec l'extension nRF Connect. L'extension nRF Connect est un outil puissant conçu pour faciliter le développement d'applications pour les microcontrôleurs de la série nRF de Nordic Semiconductor. Cette extension intègre des fonctionnalités spécifiques pour le développement de firmware, la configuration matérielle et le débogage des dispositifs nRF. Nous avons également utilisé l'extension DeviceTree, permettant de visualiser les modifications faites sur le DeviceTree de la carte par le biais d'un overlay.

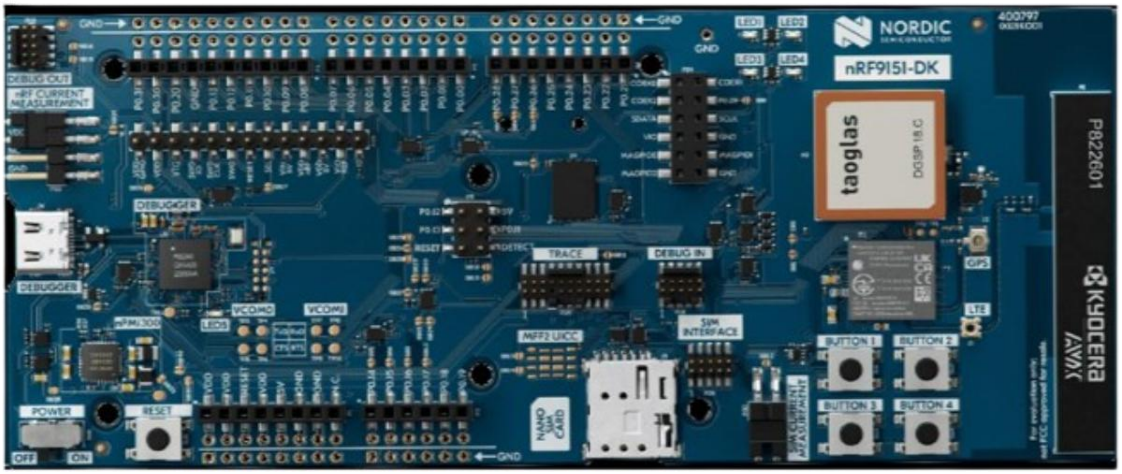

Figure 1 - nRF 9151 - DK

De plus, afin de programmer le firmware du modem du nRF9151-DK, nous avons utilisé l'outil « nRF Connect for Desktop » installé sur une distribution Ubuntu de Linux, permettant ainsi de gérer directement les installations sur le kit de développement.

2. Description du système

Lorsque l'on pense à un équipement, on pense à trois grandes fonctionnalités : la première étant le pistolet et le tir, ainsi que la détection du tir lorsque l'on est touché. Cette fonctionnalité définit le jeu du laser game et est donc essentielle à ce projet. C'est par ailleurs sur cette fonctionnalité que nous passerons le plus de temps. Par la suite, la deuxième fonctionnalité majeure est le suivi du score. Le but du jeu étant de faire le score le plus élevé, il est important d'enregistrer chaque coup au but. Enfin, la troisième fonctionnalité est la gestion des lumières. Une partie de laser game serait moins intéressante sans lasers et sans LEDs qui clignotent pour nous dire que quelqu'un nous a touchés.

C'est autour de ces trois axes que le développement de notre PFE s'est articulé.

2.1 Système de « tir » (émission – réception)

2.1.1Technologies utilisées

Pour ce qui est de l'émission et de la réception, en première approche, on imagine que l'information se transmet directement par le laser « visible » que l'on voit. En se penchant sur ce sujet, on trouve des entreprises comme « Laser Game Entreprise » annonçant une technologie 100 % laser (en comparaison à des systèmes

mélangeant technologie infrarouge (IR) et laser plus classique). On se penche alors sur la recherche de technologie laser pour transmettre l'information.

Émission - Réception LASER

On commence des tests d'émission laser à partir de la diode laser KY-008, compatible Arduino, ainsi que son module de réception, équipé d'un phototransistor et d'un circuit anti-rebond.

Les phototransistors sont des composants équivalents à un transistor bipolaire, à l'exception que la base est activée en fonction de la lumière reçue. Ce transistor peut être plus ou moins sensible à certaines longueurs d'ondes. Celui choisi est le plus sensible autour de la longueur d'onde du laser (c'est-à-dire 630 nm).

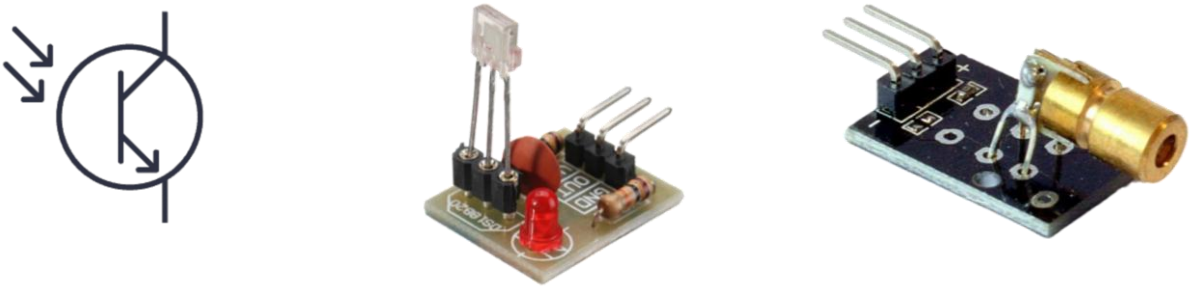

Figure 2 - Composants Lasers

Les tests ont eu des résultats peu concluants. Le premier facteur limitant est le temps de montée et de descente du phototransistor, ne permettant pas une fréquence de clignotement efficace. Bien que l'on puisse réduire la quantité d'information à envoyer, cette fréquence d'envoi est trop faible. De plus, le laser ayant une puissance <5 mW (pour des raisons de sécurité), la lumière ambiante interfère avec la détection, même à courte portée, rendant la détection du signal laser moins fiable. On arrive à des tests concluants en plaçant le laser dans une chambre sombre avec très peu de lumière, ce qui est loin d'être idéal si l'on veut étendre notre application de laser game à l'extérieur.

Cela dit, la présence du laser est essentielle d'un point de vue pratique, ne serait-ce que pour servir de pointeur laser afin d'indiquer à l'utilisateur la direction de son tir.

Émission - Réception IR

À la suite des problématiques rencontrées avec les tests laser, on décide d'essayer les technologies infrarouges (IR) déjà utilisées pour les télécommandes de télévision, par exemple. On commence alors les tests de portée avec un récepteur IR (que l'on identifiera comme TSOP38238), une télécommande IR compatible Arduino, et une carte Arduino Uno. Des bibliothèques Arduino gérant l'émission et la réception

IR existaient déjà, il était alors plus facile d'effectuer des tests de performances avec l'Arduino.

Les résultats de ces tests furent concluants : les essais avec plusieurs télécommandes (dont une télécommande de télévision) ont montré que la communication est stable sur une distance convenable. La portée de communication dépend de la puissance fournie par la diode IR. Lors des tests, la distance maximale trouvée est d'environ 10 mètres, correspondant à la longueur du bâtiment, et est jugée suffisante. Pour ce qui est des protocoles de communication, on choisit le protocole de communication NEC, dont je parlerai plus en détail ci-dessous. On choisit par la suite des composants adaptés pour implémenter cette communication. Les principales caractéristiques recherchées sont :

- Pour la diode IR : une bonne puissance d'émission maximale et une émission maximale pour une longueur d'onde correspondant à la sensibilité maximale du récepteur (autour de 950 nm pour le récepteur choisi).
- Pour le capteur IR : une sensibilité maximale autour de la longueur d'onde correspondant à la diode (autour de 900 nm pour la diode IR choisie).

On choisit alors la diode IR : MT5900-IR avec le récepteur TSOP38238. L'avantage avec ce récepteur infrarouge est qu'il intègre des composants de démodulation intégrés afin de recevoir un signal numérique (tout ou rien) à l'arrivée. Le protocole NEC utilisant un codage par distance d'impulsion avec une porteuse en PWM.

Figure 3 - Composants IR

Conclusion

Après avoir étudié et testé ces deux technologies, nous pouvons établir le comparatif suivant :

	LASER	IR
Portée	Dépend de la luminosité ambiante	Suffisante
Directivité	Très directive	Peu directive
Facilité d'implémentation de la communications	Aucunes librairies et documentations accessibles	Documentation et librairie existante
Visible	Oui	Non

Ici, on cherche une technologie fiable et facile d'implémentation au vu du peu de temps accordé à ce projet. La technologie IR serait alors appropriée. Or, l'émetteur IR n'est pas visible à l'œil nu, le joueur ne voyant pas où il tire, cela réduit l'intérêt de cette technologie. On opte alors pour une méthode hybride où l'on installe un pointeur laser afin de « guider » le tir, et la communication est assurée par la technologie IR installée parallèlement à ce pointeur afin d'assurer la précision. Pour réduire la directivité de la LED IR, un « canon » imprimé en 3D peut être envisagé.

2.2 Implémentation de l'émission et la réception

2.2.1 Hardware

Le capteur a trois broches : une pour 5V, une pour la masse, et une pour la donnée qui est active sur un signal bas. Pour ce qui est de la diode IR, une résistance de 1 kΩ est à ajouter entre l'anode et la sortie GPIO à 1,8 V de la carte.

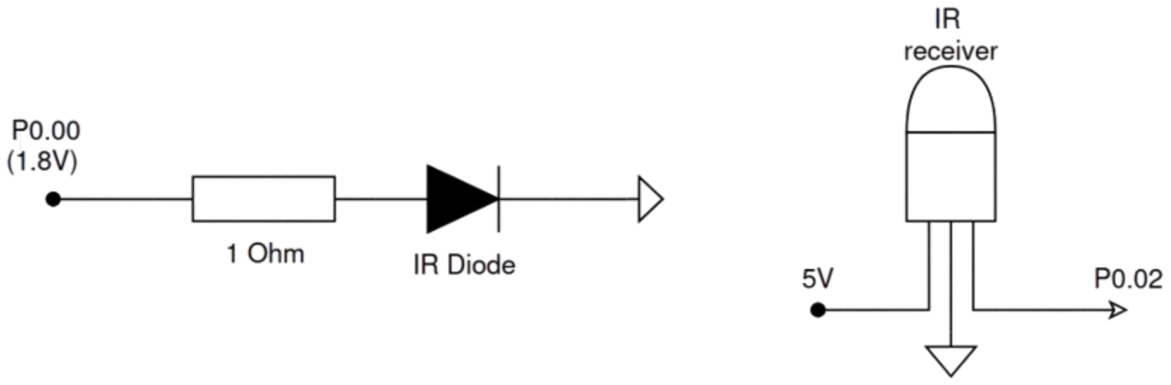

Figure 4 - Circuits IR

2.2.2 Protocole NEC

Pour réaliser la transmission IR, on prend la décision d'utiliser le protocole NEC car il est facile d'utilisation, fiable et bien documenté. Le protocole NEC est un

protocole de communication utilisant le codage par durée d'impulsion avec une porteuse de 38 kHz (PWM avec un rapport cyclique de 1/3 ou 1/4 étant conseillé). Pour coder un '0' logique, une impulsion de 560 µs de la porteuse suivie de 560 µs de pause est envoyée. Pour un '1' logique, une impulsion de 560 µs de la porteuse suivie de 1 680 µs de pause est envoyée.

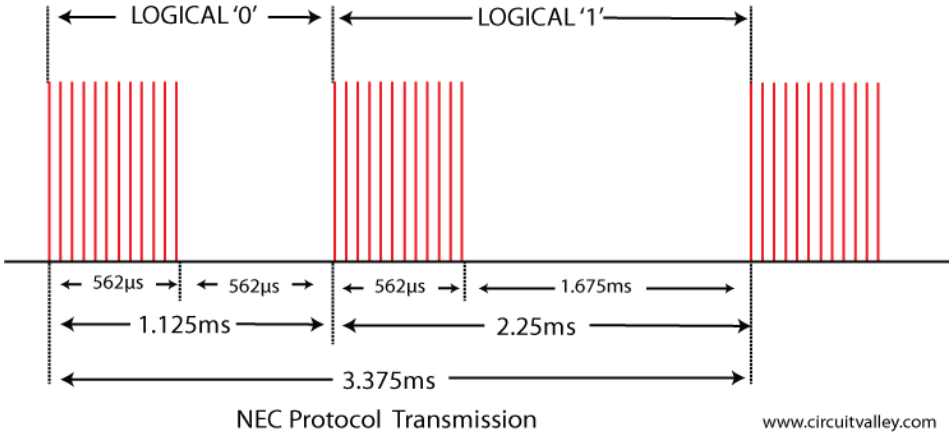

Figure 5 - NEC : exemple modulation par largeur d'impulsion

De plus, le début de l'algorithme du protocole NEC est composé d'une impulsion longue de 9 ms suivie de 4,5 ms de pause, indiquant au récepteur le début de la transmission. Pour chaque chaîne de 8 bits envoyée, il envoie aussi l'inverse logique de cette dernière, permettant ainsi d'avoir le même nombre de '1' et de '0' dans la trame. Cela crée une trame de longueur fixe et assure une meilleure fiabilité (fonctionnalité que nous n'avons pas implémentée pour avoir une meilleure quantité de données à envoyer).

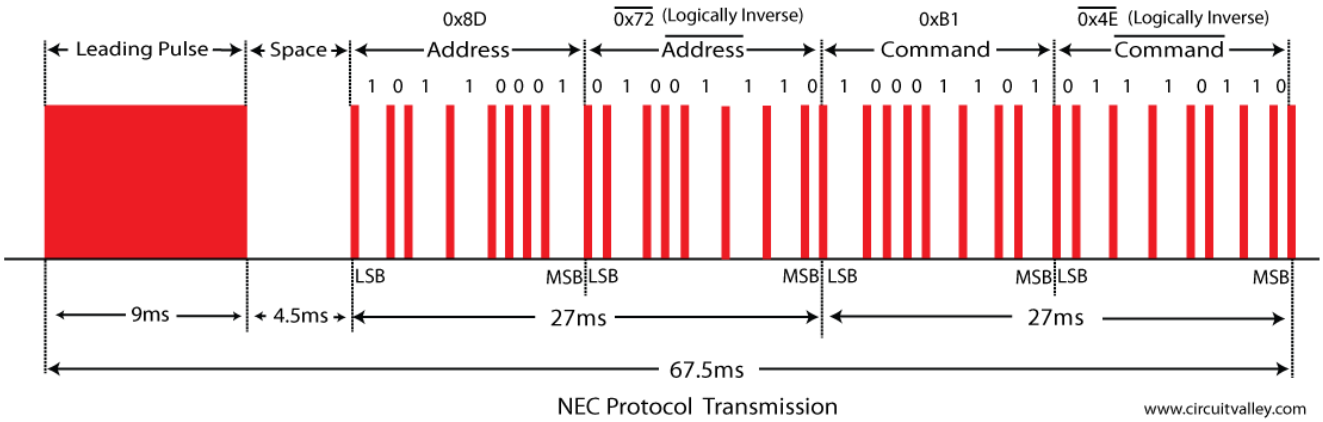

Figure 6 - Exemple de trame

La seule problématique rencontrée pour l'implémentation de ce protocole sur notre microcontrôleur est que nous n'avons pas trouvé de bibliothèque NEC en C déjà développée. Par conséquent, il a fallu en développer une par nos propres moyens. Ayant le détail de l'algorithme nécessaire à son implémentation dans la bibliothèque Arduino, il suffit d'étudier et de comprendre cette bibliothèque et de l'adapter en C pour le microcontrôleur souhaité. Cette étape a pris un certain temps mais s'est conclue par un succès. Il a fallu adapter la tolérance à cause de l'instabilité de la PWM du microcontrôleur, qui ne s'arrêtait pas et ne s'allumait pas immédiatement au moment idéal (ce qui n'a rien de surprenant), ou à cause du bruit que peut recevoir le capteur.

2.3 Gestion des lumières

Pour ce qui est de la gestion des lumières sur l'équipement, nous nous proposons d'utiliser les LEDs RGB SK6812. Ce sont des LEDs RGB composées de trois broches : une pour 5V, une pour GND, et une pour Data.

Figure 7 - LEDs RGB SK6812

La broche Data sert à commander les trois composantes R, G et B via un protocole RZ unipolaire à une fréquence de 800 kHz. Chaque composante est codée sur 8 bits et est envoyée dans l'ordre suivant : vert, rouge, bleu.

G7	G6	G5	G4	G3	G2	G1	G0	R7	R6	R5	R4
R3	R2	R1	R0	B7	B6	B5	B4	B3	B2	B1	B0

Note: high starting, in order to send data (G7 - G6 - ....B0)

Figure 8 - Protocole d'envoie des donnés pour la SK6812

Le premier obstacle était l'absence de bibliothèques fonctionnelles et adaptées pour contrôler nos LEDs. Nous avons donc été contraints de développer notre propre bibliothèque. Lors des essais, le principal problème a été la synchronisation de la modulation de largeur d'impulsion (PWM) à la bonne fréquence.

Le microcontrôleur nRF9151 offre plusieurs fréquences de PWM : 16 MHz, 8 MHz, 4 MHz, 2 MHz, 1 MHz, 500 kHz, 250 kHz et 125 kHz. Pour résoudre ce problème, nous avons eu l'idée de sélectionner une fréquence plus élevée, multiple de la fréquence souhaitée, et d'envoyer un nombre de bits plus élevé. Par exemple, en choisissant 4 MHz, qui correspond à 5 fois 800 kHz, il fallait envoyer 5 fois plus de bits. Pour envoyer '01', il fallait envoyer '0000011111'. Cependant, cette approche n'a pas fonctionné, probablement parce que les LEDs ne reconnaissaient pas les 1 et les 0 de la PWM tournant à une fréquence trop élevée.

N'ayant pas obtenu de résultats concluants, nous avons décidé de nous tourner vers des LEDs RGB à 4 broches avec une cathode commune. L'avantage de ce type de LEDs est que les composantes rouge (R), verte (G) et bleue (B) peuvent être commandées indépendamment sur trois broches GPIO différentes. Cela est idéal pour un ensemble de LEDs devant afficher la même couleur. Cependant, cela peut être un désavantage si les broches GPIO viennent à manquer, ce qui n'est pas notre cas.N'ayant pas obtenu de résultats concluants, nous avons décidé de nous tourner vers des LEDs RGB à 4 broches avec une cathode commune. L'avantage de ce type de LEDs est que les composantes rouge (R), verte (G) et bleue (B) peuvent être commandées indépendamment sur trois broches GPIO différentes. Cela est idéal pour un ensemble de LEDs devant afficher la même couleur. Cependant, cela peut être un désavantage si les broches GPIO viennent à manquer, ce qui n'est pas notre cas.

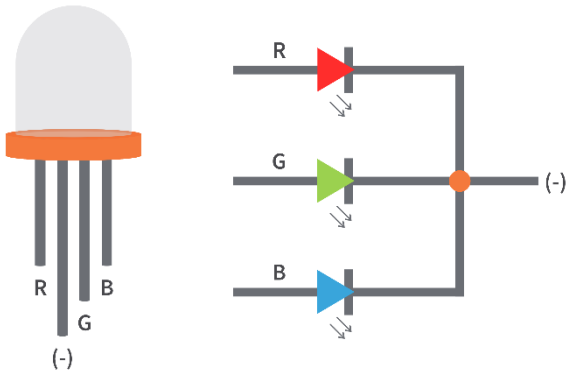

Figure 9 - Schéma de LED à cathode commune

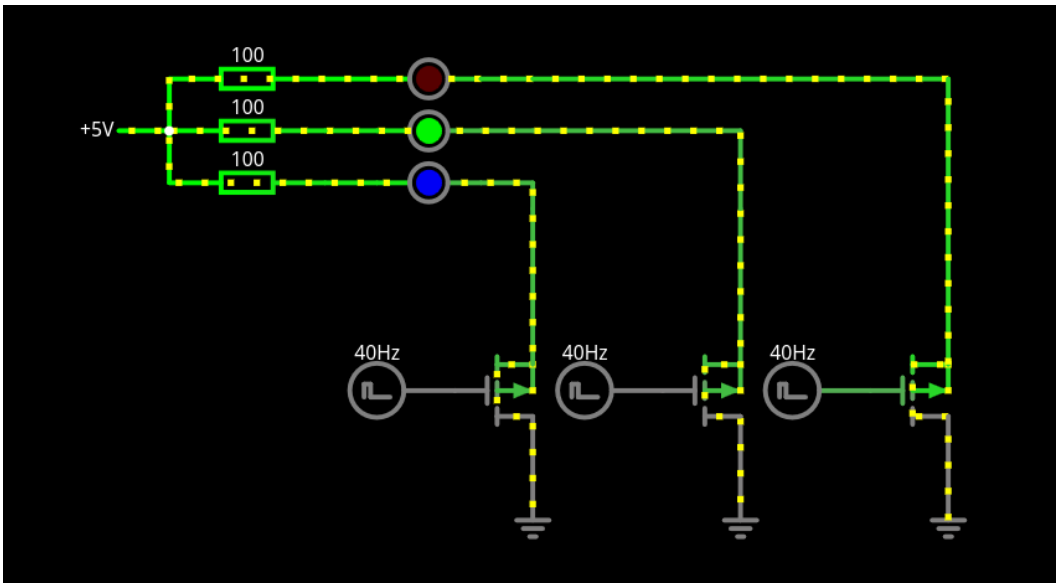

Figure 10 - Circuit contrôle de LED RGB

Dans le circuit ci-dessus, utilisé pour le prototype, on alimente les LEDs RGB en 5 V et, pour les contrôler, on utilise des transistors MOSFET à canal P. La commande à 1,8 V est reliée à la **G**rille, les LEDs à la **S**ource et la masse au **D**rain. En pratique, on utilise un transistor P-MOS IRF9Z34N pour le prototype. On est obligé d'utiliser un P-MOS dans le prototype car les sorties GPIO ne sont qu'à 1,8 V, tandis que les LEDs et la diode laser fonctionnent en 5 V. Sur la carte finale, on pourra ajouter des régulateurs de tension plutôt que des transistors afin de pallier cet obstacle.

2.4 Serveur pour le suivi de score

2.4.1 Côté Carte

L'objectif ici est de permettre la communication entre la carte microcontrôleur et un serveur distant. Le sujet du PFE spécifie l'utilisation de la technologie NR+, qui est une évolution de la 5G cellulaire. Cependant, étant donné que le NR+ est un protocole de communication relativement récent, il n'existe pas encore de drivers, de bibliothèques ou de stacks pour le piloter. En raison de cette limitation, nous avons décidé de changer de technologie et de passer à la technologie LTE-M, pour laquelle il existe déjà des bibliothèques et des exemples d'utilisation bien documentés par Nordic Semiconductor.

LTE-M est une technologie de communication sans fil conçue spécifiquement pour les dispositifs de l'Internet des objets (IoT). Basée sur la norme LTE (Long Term Evolution), elle offre plusieurs avantages pour les applications IoT. LTE-M est particulièrement efficace en termes d'énergie, ce qui est crucial pour les dispositifs IoT qui doivent souvent fonctionner pendant de longues périodes avec des batteries limitées. Elle bénéficie également de la couverture étendue des réseaux LTE existants,

assurant une connectivité fiable même dans des zones rurales ou mal desservies. La technologie utilise une bande passante réduite par rapport aux réseaux LTE traditionnels, ce qui permet de réduire la consommation d'énergie et les coûts de données. Malgré cette réduction de bande passante, LTE-M offre une latence relativement faible, essentielle pour les applications nécessitant une réponse rapide. Les modules LTE-M sont généralement moins chers que les modules LTE traditionnels, réduisant ainsi les coûts de déploiement pour les solutions IoT.

De plus, LTE-M est compatible avec les infrastructures LTE existantes, facilitant son déploiement et son intégration dans les réseaux cellulaires actuels. Elle est optimisée pour les dispositifs IoT, offrant des fonctionnalités spécifiques comme le support des messages courts (SMS) et des alertes de données. Comme toutes les technologies basées sur LTE, LTE-M bénéficie des mêmes niveaux de sécurité et de chiffrement, assurant ainsi la protection des données transmises. Elle offre également des garanties de qualité de service, essentielles pour les applications IoT nécessitant une connectivité fiable et constante.

Pour programmer la carte, nous utilisons une carte SIM IoT que nous branchons à la nRF9151. Cette carte SIM IoT est essentielle pour établir la connexion LTE-M. Nous avons choisi de nous baser sur l'exemple de nRF Connect appelé « HTTP_client » pour récupérer les fonctions essentielles nécessaires à la programmation de notre application. Cet exemple nous fournit une base solide pour envoyer des requêtes HTTP au serveur distant.

En plus de programmer notre application, il est également nécessaire d'utiliser le programmeur « nRF Connect for Desktop » pour flasher le firmware du modem de la nRF9151 DK. Ce firmware est crucial pour assurer une communication stable et efficace entre la carte et le serveur distant. Le processus de flashage du firmware permet de configurer le modem pour qu'il puisse utiliser les fonctionnalités LTE-M.

2.4.2 Côté serveur

Pour développer un serveur web capable de récupérer des données envoyées par un microcontrôleur utilisant la technologie LTE-M, nous avons opté pour l'utilisation d'un serveur privé virtuel (VPS) hébergé sur OVH, fourni par Codium. Ce VPS sert de plateforme pour recevoir les requêtes HTTP provenant de la carte microcontrôleur.

Pour la programmation du serveur, nous avons choisi Python en raison de sa simplicité et de sa flexibilité. Plus précisément, nous utilisons le framework Flask, qui facilite grandement la gestion des requêtes HTTP. Flask est particulièrement adapté pour ce type de projet en raison de sa légèreté et de sa capacité à gérer efficacement les routes et les requêtes.

L'utilisation de Python nous permet également de manipuler et de stocker les données reçues de manière aisée. Nous pouvons écrire des scripts pour traiter les données et les enregistrer dans des fichiers spécifiques, ce qui est essentiel pour

l'analyse ultérieure. Actuellement, notre application gère les requêtes HTTP GET et POST. La requête POST joue un rôle crucial dans notre système, car elle est utilisée pour envoyer les données de réception du tir au serveur. Ces données sont ensuite traitées et stockées pour une utilisation future, comme la génération de rapports ou l'analyse des performances.

4. Améliorations possible

Pour le moment, le projet n'est qu'à l'état de prototype et existe comme preuve de faisabilité. En effet, nous avons utilisé le nRF9151 DK, qui est un kit de développement servant à programmer des applications avec tous les composants nécessaires déjà installés sur la carte. Or, ce kit n'est pas voué à être mis en production en raison de son prix et de son manque de flexibilité. Ainsi, les prochaines étapes du projet sont les suivantes.

4.1 Conception de carte

Dans un premier temps, on peut commencer par la conception des cartes. On pourra utiliser le logiciel FreeCAD pour le design et le routage de celles-ci. On imagine ainsi cinq types de cartes afin de créer un système modulaire :

- **Carte dos :** Cette carte accueillera le microcontrôleur ainsi que l'antenne LTE-M du système. Elle intégrera également quelques capteurs, permettant de créer des cibles dans le dos. Pour des raisons de simplicité, c'est sur cette carte que seront placés les régulateurs de tension, afin de fournir les niveaux adaptés aux différents équipements (capteurs et émetteurs laser IR). Plusieurs connecteurs, disposés sur les bords, permettront la communication et la liaison avec les autres cartes, de manière à ce que les câbles puissent être intégrés dans la doublure d'un éventuel gilet de jeu.

- **Cartes épaules :** Ces cartes seront stratégiquement placées au niveau des épaules (avant et arrière) des joueurs. Chacune d'elles intégrera un ou plusieurs capteurs, permettant d'ajouter des cibles sur ces zones.

- **Carte plastron :** Cette carte sera placée au niveau du torse du joueur, intégrant une plus grande quantité de capteurs pour former une zone cible étendue. Elle pourra également faire office de relais pour le connecteur reliant la carte arrière à celle du pistolet.

- **Carte Pistolet :** Une carte sera placée dans le pistolet du joueur, accueillant l'émetteur laser et l'émetteur IR, installés de manière à être

le plus proche possible l'un de l'autre et parfaitement alignés sur le même axe. Le laser servira de pointeur pour le joueur, garantissant que le tir en IR suive avec précision l'axe du laser. Une impression 3D pourra être envisagée pour verrouiller mécaniquement ces deux composants.

Toutes ces cartes seront équipées d'un nombre suffisant de LEDs RGB 4 broches, comme décrit précédemment, afin d'informer le joueur et les joueurs à proximité de son état (notamment s'il a été touché par un autre joueur).

4.2 Modélisation 3D

Certaines pièces de l'équipement de laser game peuvent être imprimées en 3D pour assurer une contrainte mécanique efficace des différentes cartes et composants du système. L'objectif est d'intégrer ce système dans un gilet, permettant ainsi au joueur de porter l'équipement de manière confortable et pratique. Pour relier les différentes parties du système, nous utilisons des connecteurs solides mais interchangeables, ce qui améliore la modularité et la flexibilité de l'ensemble

Notamment, des « canons » pour le pistolet ont été conçus et imprimés en 3D pour optimiser la directivité de l'émetteur infrarouge (IR). Cette approche permet non seulement de renforcer la précision de la transmission, mais aussi de garantir une meilleure intégration des composants dans l'équipement final.

5. Sources

- Protocole NEC :
  https://si.blaisepascal.fr/1t-protocole-nec/

- nrf-9151 :
  https://docs.nordicsemi.com/bundle/ps_nrf9151/page/nRF9151_html5_keyfeatures.html

- Photodiodes & Photos transistors :
  https://www.digikey.fr/en/articles/the-basics-of-photodiodes-and-phototransistors-and-how-to-apply-them

- DECT NR+ :
  https://docs.nordicsemi.com/bundle/ncs-3.0.0/page/nrf/protocols/dect/index.html

- Capteur TSOP38438 :
  https://www.digikey.fr/en/products/detail/vishay-semiconductor-opto-division/TSOP38238/1681362

- Documentation Capteur TSOP38438 :
  https://www.vishay.com/docs/82491/tsop382.pdf

- Diode IR :
  https://www.digikey.fr/fr/products/detail/marktech-optoelectronics/MT5900-IR/5956247

- Documentation Diode IR :
  https://specs.marktechopto.com/pdf/products/datasheet/MT5900-IR.pdf

- Documentation Flask :
  https://flask.palletsprojects.com/en/stable/

- Github du projet :
  https://github.com/PrillBlizzard/Laser_Game_NR-_Codium